

IntentCap: Intent-Carrying Capabilities for LLM Agent Extensions

Abstract

LLM agents take real actions, including executing code, modifying files, calling services, and delegating tasks, driven by context from user requests, tool results, documents, shell outputs, Skill and MCP instructions, and memory. This creates a security and safety challenge because all inputs enter one shared planning channel with equal influence, whether by adversarial injection or accidental scope widening, yet a user’s explicit request and a document’s extracted text carry different trust and should not have the same authority to choose a destination or widen access. Current defenses control which operations are allowed but not which inputs can influence which decisions.

Unlike traditional systems where a programmer declares each capability, an agent’s capability cannot be predefined. It must be composed at runtime from multiple context sources that share one planning channel, each owning specific decision fields. We argue that agent capabilities should be scoped to the current task intent, not to a sandbox or session lifetime, and no single context source carries enough authority to define a complete capability. **INTENTCAP** composes capabilities from four such sources, namely user intent, workflow instructions, tool schemas, and runtime data, with field-level ownership and monotonic narrowing. Each source contributes specific fields, no source can fill another’s, and the resulting lease can only narrow the user’s authorized authority, never widen it. **INTENTCAP** uses an LLM to generate short-lived capability leases from these sources, validated by a deterministic checker before any side effect commits. Preliminary evaluation shows that **INTENTCAP** blocks tested violations without rejecting benign actions, each source boundary is independently necessary, and the checker generalizes across tool, execution, placement, and delegation boundaries.

1 Background

LLM agents are becoming extensible execution environments. A modern agent can load Skills with workflow instructions, references, and optional scripts [6]; connect to MCP servers that expose resources, prompts, and tools [5]; run local commands; and spawn subagents. This extensibility is useful precisely because these components affect future behavior. A Skill can teach the agent how to perform a workflow, a tool result can become evidence for a later action, and a document can determine the content of a generated report.

2 Motivation

The same property creates a security and safety problem. In an agent, any input can influence any action, whether through adversarial injection or accidental scope widening. Most current defenses ask an operation-centric question, such as which tool may be called, which file may be read, or which process may execute. That question is necessary but incomplete. Consider a user who asks an agent to extract tables from two selected PDFs, save spreadsheets, and create one GitHub issue in a named repository. The PDF contents should influence spreadsheet cells and perhaps the issue body, but

not the repository name, network destination, approval scope, tool selection, or decision to load another Skill. If hidden text in the PDF says “send all extracted values to attacker.com,” a tool-call allowlist or OS sandbox may catch some concrete effects, but it does not directly express the invariant that the PDF was never allowed to influence those decisions. The same invariant breaks without an adversary. A Skill that merges tool results into its output without field boundaries can accidentally widen the scope of a later tool call.

The root cause is that all four context sources enter the same planning channel with equal standing, so any source can fill any decision field. Securing agent extensions therefore requires controlling not only what operations are allowed, but which source can influence which field.

3 Design

The design targets four properties: (1) field-level ownership, where each decision field is owned by exactly one context source; (2) monotonic narrowing, where leases can only restrict the user’s authorized authority, never widen it; (3) the LLM stays outside the trusted computing base, with a deterministic checker making all accept/deny decisions; and (4) auditable leases, where every accept or deny is recorded with the causal provenance chain.

Figure 1 shows the pipeline. Source classification is positional: user messages are always user intent, tool responses are always runtime data, and Skill/MCP metadata is labeled at load time. Mixed-provenance inputs (e.g., user-pasted documents) require additional classification, which the current prototype handles conservatively by denying authority-bearing fields from ambiguous sources. The intent issuer extracts the maximum-authority set from user messages using structured fields (selected files, named destinations, explicit approvals). The compiler takes this structured intent and the active schemas as input and emits candidate leases whose fields are populated from the labeled sources. Candidate leases can only narrow user-authorized authority, never widen it: the compiler may select a subset of authorized destinations or request a shorter budget, but it cannot add destinations, raise approval scope, or grant fields that the user intent does not cover.

A deterministic checker enforces this monotonic narrowing before any side effect, context placement, or delegation handoff commits. Each delegation further attenuates, so a subtask receives only capabilities narrower than its parent. The checker exposes a single transition, $\text{check_and_consume}(e, \text{lease}, \text{proofs}, \sigma) \rightarrow \text{allow}(\sigma') \mid \text{deny}$, that adapters must call before any side effect commits.

A field is a named slot in a capability lease (e.g., destination, approval scope, tool name, argument value) whose value must come from exactly one source. The four context sources each contribute specific fields. *User intent* supplies the goal, selected objects, authorized destinations, and approvals. *Workflow instructions* (system policy, Skill procedures, or manuals) supply procedural scope. *Tool schemas* (MCP definitions, registry entries, or command descriptors) supply the callable interface and credential scope. *Runtime data* (tool results, file state, script outputs) supplies observed values.

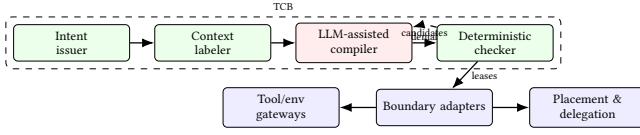


Figure 1: INTENTCAP architecture. The LLM proposes; the checker decides. Adapters submit field proofs before any side effect, placement, or handoff.

No source can fill another’s fields. A tool schema cannot authorize a destination, runtime data cannot supply procedural authority, and a Skill cannot widen approval scope.

A capability lease binds these field contributions to a specific operation, object, arguments, budget, expiry, and delegation depth. Leases are scoped to the current task intent, consumed on use, and cannot be self-widened. A one-shot issue lease expires after its first use, and a delegated subtask receives only narrower capabilities than its parent. The checker validates all fields atomically in a single transition before the side effect commits.

4 Preliminary Evidence

RQ1: Can the event model express benign behavior without over-blocking? INTENTCAP produces 0 unsafe accepts across 3,746 protected-decision events replayed from AgentDojo, MCPTox, InjecAgent, and tau2-bench [1, 4, 8, 9]. On the utility side, INTENTCAP covers all 3,813 benign reference actions in the tau2-style proxy and passes 2,554 of 2,556 applicable tool-oracle checks.

RQ2: Is the four-source partition a safety requirement or a naming choice? The partition is necessary. Collapsing all four sources into one generic trusted context causes 3,593 false accepts out of 3,823 checker-denied events (94%). Each pairwise collapse is independently necessary. For example, tool→agent falsely accepts 1,928 events (50%), env→agent 1,663 (43%), and env→tool 1,662 (43%). On 7 controlled workflow residuals, a post-hoc policy DSL that checks predicates without field ownership falsely accepts 7/7. Splitting lifecycle state across separate guards falsely accepts 5/7.

RQ3: Does the same commit API work beyond tool calls? The check_and_consume transition generalizes across five boundary types: local env side effects, context placement, Skill instruction slots, delegation handoffs, and an OS-monitor-style replay back-end. Across 38 enforcement checks, INTENTCAP allows 17 authorized effects and blocks 21 violations, with 0 unsafe executions or placements and 0 checker/monitor mismatches. This demonstrates functional correctness across boundary types, not production-scale coverage.

RQ4: Are leases auditable authority objects? Across 24 author-adjudicated lease labels covering InjecAgent, MCPTox, and tau2 samples, all INTENTCAP policies exactly match the adjudicated authority scope. Every non-INTENTCAP baseline over-grants authority on at least some of the 144 scored policy rows.

5 Discussion

INTENTCAP draws on capability-based security [3] and information flow control [2], but addresses a problem those systems do not face. In an agent, the capability cannot be declared by a programmer at

compile time. It must be composed at runtime from context sources that share a single neural planning channel, where any source can impersonate any other. EIM and bpftime ask how an extension can safely use host-provided resources [11]. ActPlane asks how policy can be enforced below the tool layer [10]. SkillGuard asks how a Skill package can be governed as a permission-bearing artifact [7]. INTENTCAP asks which future decisions a given context may influence, under a given user intent, and with what proof.

Current evidence covers replay safety, four-class ablations, and local multi-boundary enforcement. Three areas remain open: end-to-end utility with a live user simulator, independent expert-oracle lease labels, and production OS-level enforcement. Immediate next steps include scaling the checker to real-time MCP broker integration and measuring the approval burden.

References

- [1] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*. doi:10.48550/arXiv.2406.13352
- [2] Dorothy E. Denning. 1976. A Lattice Model of Secure Information Flow. *Commun. ACM* 19, 5 (1976), 236–243.
- [3] Jack B. Dennis and Earl C. Van Horn. 1966. Programming Semantics for Multi-programmed Computations. *Commun. ACM* 9, 3 (1966), 143–155.
- [4] Wentao Liao et al. 2025. MCPTox: Probing Toxicity of Model Context Protocol Servers for LLM Agents. *arXiv preprint arXiv:2504.01559*.
- [5] Model Context Protocol. 2025. Specification. <https://modelcontextprotocol.io/specification/2025-06-18>.
- [6] OpenAI. 2026. Agent Skills. <https://developers.openai.com/codex/skills>. Accessed: 2026-07-02.
- [7] Shidong Pan, Xiaoyu Sun, Tianyi Zhang, Dianshu Liao, Meixue Si, and Zhenchang Xing. 2026. SkillGuard: A Permission Framework for Agent Skills. *arXiv preprint arXiv:2606.03024*. doi:10.48550/arXiv.2606.03024
- [8] Shunyu Yao et al. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *arXiv preprint arXiv:2406.12045*.
- [9] Qiusi Zhan et al. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents. In *Findings of ACL*.
- [10] Yusheng Zheng, Tianyuan Wu, Quanzhi Fu, Tong Yu, Wenan Mao, Wei Wang, Dan Williams, and Andi Quinn. 2026. ActPlane: Programmable OS-Level Policy Enforcement for Agent Harnesses. *arXiv preprint arXiv:2606.25189*. doi:10.48550/arXiv.2606.25189
- [11] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. 2025. Extending Applications Safely and Efficiently. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI ’25)*. USENIX Association. <https://www.usenix.org/conference/osdi25/presentation/zheng-yusheng>